People's Democratic Republic of Algeria
University of Sciences and Technology Houari Boumediene
Computer Science Faculty



Reminders on the Basics of Algorithms

Data Structures and Algorithms (L2 - Computer Science)

Edited by: Dr. B. BESSAA

Algorithm definition

an algorithm is a set of **well-defined** instructions to **solve** a particular problem. It takes a **set of input**(s) and **produces** the desired **output**.

An algorithm is any **well-defined** computational procedure that takes some value, or set of values, as **input** and **produces** some value, or set of values, as **output**. An algorithm is thus a **sequence of computational steps** that **transform** the input into the output. (Cormen, Leiserson, Rivert).

An algorithm is a set of **elementary** computational operations **organized** according to **precise rules** in order to **solve** a given problem. For each piece of **data** in the problem, it **returns** a solution after a **finite** number of operations.(Beauquier, Berstel, Chrétienne)

An algorithm is a **finite** and **unambiguous** sequence of operations or instructions to solve a **class of problems**

Properties of an algorithm

1- Validity: the validity of an algorithm is its ability to perform exactly the task for which it was designed.

2- Robustness: robust algorithm is designed to perform **reliably** and **accurately** under a **variety** of conditions, including **unexpected** or **adverse** circumstances without breaking or producing incorrect results. (different instances, edge cases, size of data, etc.)

3- Reusability: The reusability of an algorithm is its ability be used again in **different** contexts or for solving multiple, **related** problems **without significant** modification (generalization, modularity, parameterization).

4- Complexity: The complexity of an algorithm is the amount of resources needed to process inputs. The main resources measured are time (number of instructions used) and space (amount of memory space required).

Data Structures and Algorithms

Algorithmics

Algorithmics is the systematic study of the design and analysis of algorithms. Algorithmics is interested in the art of building algorithms and evaluate their properties (validity, robusteness, reutilisability, complexity and efficiency).
Algorithmics allows us to move from a problem to be solved to the problem-solving approach.

The problem-solving appraoch

Analyze the problem : obtain a description of the problem.

Define the data: their features, their types.

Define the results : their features, their types.

Define the relationship between results and data: how to move from data to results?

Data Structures

Data structure is a way to store and organize **data** so that it can be used **efficiently** for storing, processing, and retrieving. A data structure is a concrete implementation of an **Abstract Data Type (ADT)**.

" ***I will, in fact, claim that the difference between a bad programmer and a good one is whether he considers his code or his data structures more important.***
Bad programmers worry about the code.
Good programmers worry about data structures and their relationships. "

— Linus Torvalds (creator of Linux)



General structure of an algorithm

Header Part

Declaration part

Action Part

```
Algorithm  <nameAlgo> ;

Const  <IdObj> = <ValueObj> ;
Type  <IdObj> = <idType> ;
Var  <IdObj> : <TypeObj> ;

Begin
    <Action1> ;
    <Action2> ;
        ---
    <ActionN> ;
End.
```

Actions

Assignment :

<IdObj> ← <Expression> ;

Control Structures :

Selection Statements (conditional statements)

If <Condition> **Then** <Block Action> **Endif;** **(basic form)**

If <Condition> **Then** <Block Action1> **Else** <Block Action2> **Endif;**

Multiple-conditions structure (Switch statements)

Case <Expression> **Of**
<Val1> **:** <Block Action1>;
<Val2> **:** <Block Action2>;

<ValN> **:** <Block ActionN>;
Else <Block OtherActions> ;
EndCase;

Iterative structures (loops) :

While <Condition> **Do** <Block Action> **Endwhile;**

Repeat <Block Action> **Until** <Condition> **;**

For <IdCompt> ← <IV> **To** <FV> **Do** <Block Action> **Endfor;**

For <IdCompt> **From** <IV> **To** <FV> **Do** <Block Action> **Endfor;**

Parameterized Functions (Subroutines) :

Functions :

Function <FunctionName> (<formal parameters >) : <TypeResult>**;**
Var < Local variables declaration > **;**
Begin
    <Block Action> **;**
    <FunctionName> ← <Expression> **;**
End;

```
Function <FunctionName> (<formal parameters >) : <TypeResult>;
Var < Local variables declaration > ;
Begin
        <Block Action> ;
         return <Expression> ;
 End;
```

Procedures:

```
Procedure <ProcedureName> (< formal parameters >) ;
Var < Local variables declaration > ;
Begin
        <Block Action> ;
End;
```

Call of a parameterized function:

<FonctionName> (<Effective parameters>) Acts like : **Expression**

<ProcedureName> (< Effective parameters>) Acts like : **Action**

Parameter Passing Modes:

Pass by value: in this case the subroutine uses copies of the data passed as parameters. Once completed, the **original** values **remain unchanged**.

Effective Parameters are handled as local variables.

Pass by reference (address): in this case the subroutine uses the **original** data. Once complete, the **original** variables are **affected by any changes**.

Effective Parameters are handled as (global variables).

Data Structures

Basic Types (Simple): Integer, Real, Charactere, Boolean

Array: a collection of same data types elements.

Record : a collection of fields, possibly of different data types

File: Representation of items in secondary memory.

List: Dynamic memory allocation (series of nodes)

Syllabus

1- Analysis of Algorithms (complexity)

2- Dynamic Allocation and Linked List (SLL, DLL, CLL)

3- Stacks and Queues

4- Recursion

5- Trees

6- Hash Tables

Data Structures and Algorithms

2 - 3 - 5 - 6 1 - 4

Bibliography

Reema Thareja « Data Structures Using C» Oxford University Press, 2nd Edition, 2014.

Tim Roughgarden « Algorithms Illuminated: The basics», 2017 (https://www.algorithmsilluminated.org/won1sample.pdf)

Michel Divay. « Algorithmes et structures de données génériques : Cours et exercices corrigés en langage C » Ed. DUNOD 2eme édition 2004.

T.H. Cormen, C. Leiserson, R. Rivest, C. Stein. « Introduction to algorithms » The MIT Press Cambridge, 4th Edition 2022.

Debdutta Pal, Suman Halder « Data Structure and Algorithm with C» Alpha Science International Ltd., 2018.

Brahim BESSAA « Data Structures and Algorithms: Exercices and Problems with solution, Lecture notes and C language programs» Ed. Pages Bleues 2024.

